5-2 · Spark 마스터-워커 구축 · 실습 1.5H

Docker Compose로 마스터-워커 클러스터 구축

마스터 1 + 워커 2 를 compose 하나로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① 히어독으로 compose 파일을 쓰고 마스터 1 + 워커 2 를 띄운다

② 로그와 마스터 UI 에서 워커 등록(ALIVE)을 확인한다

③ --scale 로 워커를 늘렸다 줄이고 hostname 을 비교한다

④ 마스터를 일부러 끄고, 워커가 스스로 돌아오는 것을 본다

RAM 16GB 환경이면 워커를 1개로 — 마스터-워커 구조와 제출 흐름은 같습니다

실습 흐름

여덟 단계 — ~/data-eng/week5 한 폴더에서

4주차 컨테이너가 8080 을 쓰고 있으면 먼저 내려 둡니다 — 볼륨은 남습니다

1·2. 준비	3. 기동	4·5. 로그·UI	6·7. 스케일	8. 실패·복구
포트 비우기 compose 히어독	config → up -d → ps	등록 기록 localhost:8080	--scale 3 → 2 hostname 비교	마스터 stop → start

2단계

compose 파일 — 서비스 두 개, 핵심 줄 여덟

줄	뜻
image: apache/spark:3.5.3	마스터·워커 같은 이미지
...deploy.master.Master	마스터 — Master 클래스 실행
--host spark-master	주소를 IP 대신 이름으로 알림
8080:8080 · 7077:7077	마스터 웹 UI · 접수 포트
...deploy.worker.Worker	워커 — 첫 인자로 마스터 주소
spark://spark-master:7077	마스터 주소 = 서비스 이름
--cores 2 --memory 2g	이 워커가 내놓을 자원
deploy: replicas: 2	워커 2개 (16GB 면 1개)

--host 를 빼도 돌지만 UI URL·워커 로그가 spark://172.x.x.x:7077 로 나온다(실측)

1·2·3·4단계

핵심 명령 — 비우고 · 쓰고 · 띄우고 · 로그

포트 비우기	cd ~/lakehouse && docker compose stop
파일 만들기	cat > docker-compose.yml << 'EOF'
문법 확인	docker compose config
기동 · 상태	docker compose up -d · docker compose ps
마스터 로그	docker compose logs spark-master
워커 로그	docker compose logs spark-worker

로그에 grep -i "registering worker"(마스터) · "successfully registered"(워커)를 붙이면 등록 기록만 남는다

3·4단계

기동 뒤 확인 — ps 와 등록 로그

1. **up -d — 세 컨테이너 Started**
   spark-master · week5-spark-worker-1 · 2 — ps 에 셋 다 Up

2. **마스터 로그 — 등록 두 줄**
   Registering worker 172.x:포트 with 2 cores, 2.0 GiB RAM

3. **워커 로그 — 워커마다**
   Connecting to master spark-master:7077... → Successfully registered

up 직후 몇 초는

로그가 아직 없을 수 있음

5단계

마스터 UI(localhost:8080) — 네 곳을 읽는다

1. **URL: spark://spark-master:7077**
   다음 1H 에 잡을 제출할 주소 — --host 덕분에 이름으로
2. **Alive Workers: 2**
   Workers 표에 워커 두 줄이 ALIVE
3. **Cores in use: 4 Total, 0 Used**
   잡이 없어 Used 는 0 — Total 이 오늘 잡이 쓸 상한
4. **Memory in use: 4.0 GiB Total**
   Running / Completed Applications 는 아직 비어 있음

> **Total 은 워커가 내놓은 자원의 합 — --cores 2 --memory 2g × 워커 2**

6·7단계

늘렸다 줄이고 — 정말 여러 대인지 본다

1. **--scale spark-worker=3**
   worker-3 Started — Alive Workers 3 · Cores 6 Total · 6.0 GiB

2. **--scale spark-worker=2**
   Alive 2 · Cores 4 Total — 지운 워커는 표에 DEAD 로 남는다

3. **hostname 세 줄**
   마스터와 워커 둘이 서로 다른 12자리 — 컨테이너 ID 가 곧 호스트명

DEAD 로 남은 워커

Alive 수에서만 빠진다

8단계 — 일부러 실패시키기

마스터를 껐다 켜면 — 워커는 스스로 돌아온다

stop spark-master — 실패

워커는 죽지 않고 마스터를 계속 다시 찾는다

compose stop	마스터 UI 000	워커 로그
sleep 15	접속 안 됨	Connection refused Retrying (attempt # N)

VS

start spark-master — 복구

손대지 않아도 30초 안에 워커가 다시 등록

compose start	재등록 7초 · 21초	마스터 UI 200
sleep 30	워커마다 간격 다름	ALIVE 두 줄

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. docker compose ps 에 마스터 1 · 워커 2 가 모두 Up 이다
2. 워커 로그에 Successfully registered with master 가 있다
3. UI 가 Alive Workers 2 · Cores 4 Total, 0 Used 다
4. 줄인 뒤 지운 워커는 DEAD 로 남고, hostname 은 서로 다르다
5. 마스터가 꺼진 동안은 새 잡을 못 낸다 — 그래서 운영은 HA

5-2 · 실습 1.5H 마무리

다음은 spark-submit 실습 (1H)

마스터 컨테이너 안의 spark-submit 으로 예제와 직접 쓴 스크립트를 제출하고, 마스터 UI 에서 실행을 추적합니다

- ✓ **pi.py 예제 제출 — Pi is roughly 3.14...**
- ✓ **first_job.py 제출 — master 값 확인**
- ✓ **--total-executor-cores 로 자원 제한**
- ✓ **메모리를 과하게 요구해 WAITING 재현**

가져갈 것

클러스터 Up 상태 · 마스터 UI 8080